ScriptLens v3.0 — Unconventional Testing Framework & Narrative Sensitivity Analysis

Executive Summary: Why Standard Testing Fails Narrative Software

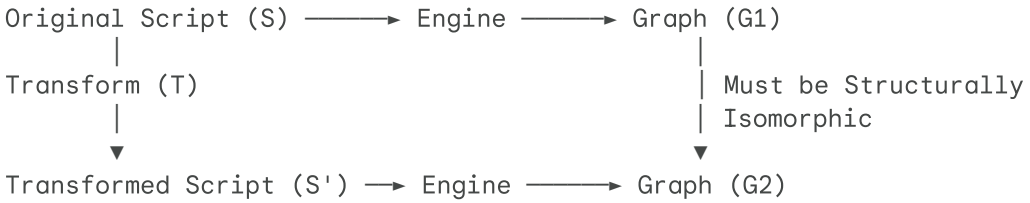
Standard software testing checks for syntax, HTTP 200 responses, and database integrity. However, **ScriptLens operates on story logic and human narrative perception**.

Because ScriptLens translates fuzzy natural language (Fountain/PDF text) into deterministic Directed Acyclic Graphs (DAGs), traditional unit tests will miss semantic drift, structural over-sensitivity, and false-positive cascades.

This document outlines **5 unconventional, domain-specific testing methodologies** designed to stress-test the story logic, NLP parser resilience, graph integrity, and human intuition alignment of ScriptLens.

1. Metamorphic Testing (Invariance & Symmetry Fuzzing)

Metamorphic testing validates system properties where exact ground truth is unknown, but relationships between outputs of modified inputs are strictly defined.



1.1 Entity Swapping (Isomorphism Test)

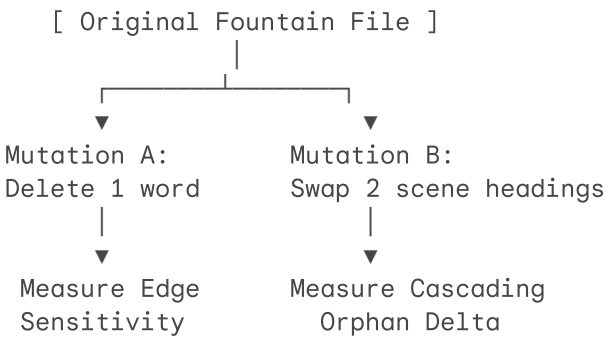
- **Hypothesis:** Replacing character or object names consistently across a script must **never** alter the topology, edge weights, or orphan status of the story graph.
- **Method:**
  1. Take a screenplay S. Run `analyze_structure(S)` to produce graph G_1.
  2. Replace all instances of `ALICE` with `CHARACTER_X` and `GUN` with `OBJECT_Y`.
  3. Run `analyze_structure(S')` to produce graph G_2.
- **Assertion:** G_1 and G_2 must be **topologically isomorphic**. Adjacency matrices must match 100%. If they differ, the NLP engine is leaking hardcoded rules, state, or casing biases.

1.2 Time Reversal (Temporal Inversion Test)

- **Hypothesis:** Since narrative flow is forward in time, reversing the chronological order of scenes should completely destroy downstream dependencies without causing cycles or engine crashes.
- **Method:** Reverse the scene order ($S_{\text{rev}} = [Scene_N, Scene_{N-1}, \ldots, Scene_1]$).
- **Assertion:** The resulting graph must have **0 downstream payoff edges** (or all forward edges must collapse into orphans). The engine must enforce the directed acyclic rule ($A \to B \implies A < B$) without throwing recursion or index errors.

2. Narrative Chaos Engineering (Graph Mutation Testing)

Borrowing from Netflix's Chaos Monkey, **Story Chaos Engineering** introduces micro-mutations into scripts to measure how sensitive or brittle the dependency graph is.



2.1 The "Single-Word Entropy" Test

- **Method:** Automatically iterate through a screenplay, deleting **one noun or named entity at a time** across 500 test runs, and measure the delta in graph edges.
- **Metric:** Compute the **Edge Sensitivity Coefficient**:

$$\text{Sensitivity} = \frac{\Delta \text{Edges}}{\Delta \text{Words}}$$

- **Goal:** Detect hyper-sensitive nodes (where removing a single typo causes 20 scenes to un-link) vs under-sensitive nodes (where deleting a major action block changes zero edges).

2.2 Scene Shuffling (Permutation Stability)

- **Method:** Swap two adjacent scenes ($Scene_i \leftrightarrow Scene_{i+1}$) that share no common entities.
- **Assertion:** The global graph state outside of $Scene_i$ and $Scene_{i+1}$ must remain 100% unchanged. This verifies that local edits cause strictly **localized graph mutations**.

3. Human-in-the-Loop Alignment & Intuition Benchmarking

A graph that is mathematically sound but narrative-wise absurd is a failure. We must benchmark ScriptLens against human story intuition.

3.1 The "Blind Reader" Sensitivity Matrix

- **Method:**
  1. Take 10 well-known screenplays.
  2. Have 3 human script readers independently list the top 5 "vital scenes that cannot be cut without breaking the story" (Ground Truth H).
  3. Run ScriptLens `get_delete_impact()` across all scenes and rank scenes by total downstream impact (Engine Rank E).
  4. Calculate **Spearman's Rank Correlation Coefficient (ρ)** between H and E.

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

- **Target:** Achieve $\rho \geq 0.75$, demonstrating high correlation between the engine's algorithmic risk rank and human narrative judgment.

3.2 The "Garbage-In" Graceful Degradation Test

- **Method:** Take a valid script and progressively inject OCR noise, missing punctuation, lowercased sluglines, and random character dialogue corruptions at intervals of 5%, 10%, 25%, and 50%.
- **Goal:** Plot the **Orphan Drift Curve**. The engine should degrade gracefully (gradually transitioning to `limited` mode or dropping edges) rather than failing catastrophically or producing wildly unpredictable orphan counts.

4. Synthetic Micro-Corpora & Property-Based Fuzzing

Instead of relying solely on full-length Hollywood PDFs, use programmatic generation to test extreme narrative edge cases in micro-scripts.

4.1 Programmatic "Chekhov's Gun" Generator

Generate thousands of 3-scene micro-scripts using `Hypothesis` or custom Python scripts:

```
INT. ROOM A - DAY
ALICE finds a {PROP_NAME}.

INT. ROOM B - DAY
{FILLER_ACTION}

INT. ROOM C - NIGHT
ALICE uses the {PROP_NAME}.
```

- **Variables Fuzzed:**
  - Distance between Scene 1 and Scene 3 (1 to 100 intervening scenes).
  - Naming variations: `a heavy brass key`, `the brass key`, `that same key`, `it`.
  - Formatting: UPPERCASE, lowercase, buried in long dialogue, buried in parentheticals.
- **Assertion:** The engine must reliably construct edge $Scene_1 \rightarrow Scene_3$ across all valid linguistic variations of `{PROP_NAME}`.

4.2 The "Infinite Echo" Loop Test

- **Scenario:** A screenplay where Scene 1 introduces 50 props, and Scenes 2 through 100 each re-reference all 50 props.
- **Goal:** Test graph density limits. Verifies that the $O(N^2)$ or $O(E)$ memory and time complexity does not cause CPU spikes or memory leaks during `simulate/cut` execution.

5. Client Memory Pressure & DOM Thrashing Analysis

Since ScriptLens targets screenwriters running **4 GB laptops with 30+ browser tabs open**, testing must go beyond normal browser performance checks.

5.1 The "Coffee Shop" Low-Memory Stress Benchmark

```
Simulated Constraints:
├── 4x CPU Throttling
```

```
├── 512 MB Tab Memory Hard Limit
└── Continuous Async Cut Simulations (100 iterations)
```

- **Method:**
  1. Using Puppeteer/Playwright, launch Chrome throttled to **4x CPU slowdown** and **512 MB memory limit**.
  2. Load a 180-scene script (`American Pie` or `Batman Begins`).
  3. Execute a continuous loop clicking "Simulate Cut" on every single scene sequentially (180 rapid API requests and DOM updates).
- **Metrics Tracked:**
  - **JS Heap Size Growth:** Must flatten and not leak memory over time.
  - **DOM Node Accumulation:** Must remain $< 1,500$ nodes (verifying virtual scroll recycler efficiency).
  - **Long Tasks (> 50ms):** Must be zero on the main thread during scroll.

6. Implementation Summary & Matrix

Methodology	Category	Primary Target	Value Proposition
Metamorphic Fuzzing	Semantic Invariance	NLP & Entity Extraction	Guarantees engine neutrality across names and formatting.
Story Chaos Testing	Graph Sensitivity	`SceneDependencyEngine`	Identifies hyper-sensitive or brittle narrative nodes.
Human Correlation (ρ)	Psychometric	Product Value / Trust	Ensures algorithmic risk ranks align with human readers.
Synthetic Micro-Corpora	Property-Based	Edge Resolution Logic	Isolates specific NLP parsing edge cases in millisecond tests.
Low-Memory Stressing	Client Performance	Thin Client / UX	Guarantees smooth execution on 4 GB budget laptops.